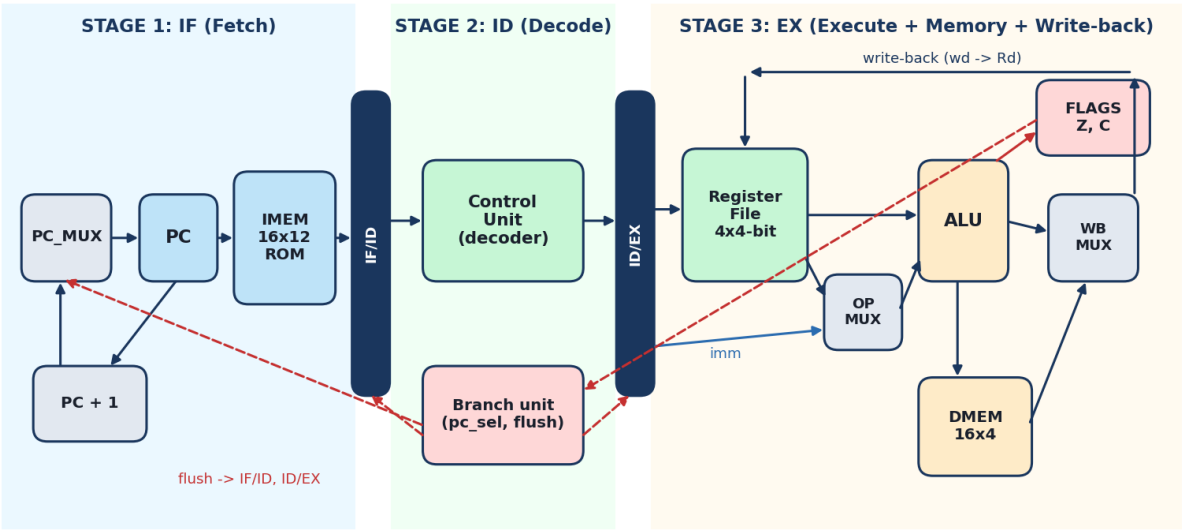
Design and Implementation of a

4-Bit RISC CPU Datapath

with a 3-Stage Pipeline, Using Verilog HDL



Project Summary Report: RTL Design, Verification and FPGA Implementation

4-bit datapath · 12-bit instructions · 4 registers · IF / ID / EX pipeline · hardwired control · 44 / 44 CPU-level checks passed · Artix-7 xc7a35tcpg236-1

Student	Student ID	Course	Instructor	Institution
Tran Duy	SE193143	[Course code / name]	[Instructor name]	[Institution]

October 2026

Contents

1. Introduction **3**

1.1 Design objective . . . 3

1.2 Architecture summary . . . 3

1.3 Toolchain and design flow . . . 3

2. RTL Architecture Analysis **4**

2.1 Functional blocks . . . 7

2.2 Datapath and control flow . . . 7

2.3 Hazard handling at RTL . . . 7

2.4 Observations from the expanded schematic . . . 8

3. Verification and Simulation Results **9**

3.1 Waveform analysis – Test 1 (demonstration program) . . . 10

3.2 Test 2 – Arithmetic and data memory . . . 12

3.3 Test 3 – Logic operations . . . 13

3.4 Test 4 – Program counter and jumps . . . 13

3.5 Test 5 – Reset and X-propagation . . . 14

3.6 Coverage and performance . . . 14

4. Resource Utilization **16**

4.1 Resource utilization . . . 16

4.2 Implemented netlist structure . . . 18

4.3 Optimisation assessment . . . 19

5. Conclusion **20**

1. Introduction

This project delivers a functional 4-bit RISC-style CPU, described in Verilog HDL at the Register-Transfer Level (RTL), organised as a 3-stage pipeline and verified end-to-end with **44 of 44 CPU-level self-checking assertions passing**. The design was taken through RTL elaboration, behavioural simulation, synthesis and implementation in AMD Vivado [version], targeting the Artix-7 device **xc7a35tcpg236-1** [board name, e.g. Basys 3].

1.1 Design objective

The goal is a complete yet minimal processor containing every essential CPU mechanism: instruction fetch, decode, execute, data-memory access, register write-back, condition flags, unconditional and conditional jumps, and instruction-level pipelining. Each functional block is an independent module, verified in isolation before integration in the top-level module `cpu_top`.

1.2 Architecture summary

Parameter	Specification
ISA style	RISC, load/store, fixed-length instructions
Datapath width	4 bits (registers, ALU, data memory)
Instruction word	12 bits: opcode [11:8], Rd [7:6], Rs [5:4], imm / addr [3:0]
Instruction set	12 instructions: ADD, SUB, AND, OR, XOR, NOT, LDI, MOV, LOAD, STORE, JMP, JZ; opcodes 1100–1111 reserved (NOP)
Register file	4 × 4-bit general-purpose registers R0–R3; 2 read ports, 1 write port
Memory organisation	Harvard: 16 × 12-bit instruction memory, 16 × 4-bit data memory
Program counter	4 bits (16-word address space)
Pipeline	3 stages: IF (fetch), ID (decode), EX (execute + memory + write-back)
Control	Hardwired: combinational decoder (ID) + branch unit (EX)
Flags	Zero (result = 0000) and Carry (after SUB, Carry = 1 means no borrow)
Throughput	1 instruction per clock after a 2-cycle fill; 2-cycle penalty per taken jump

Table 1. Key specifications of the CPU

1.3 Toolchain and design flow

All work was carried out in Vivado: RTL Analysis (elaborated schematic), the Vivado Simulator (XSim) for behavioural simulation, then Synthesis and Implementation with the Utilization report. Verification uses one CPU-level testbench, `tb_cpu`, which runs four program images (`program.mem`, `prog_arith.mem`, `prog_logic.mem`, `prog_jump.mem`) plus a reset scenario. It is complemented by the module-level testbenches `tb_alu`, `tb_register_file` and `tb_control_unit`.

2. RTL Architecture Analysis

The elaborated RTL schematic confirms a clean three-stage partition. Two pipeline registers, IR (IF/ID) and ID_EX (ID/EX), separate the fetch logic, the decode logic and a single execute stage that owns the register file, ALU, data memory, flags and next-PC selection. Figure 1 gives the conceptual block view; Figures 2 and 3 are the Vivado RTL Analysis schematics.

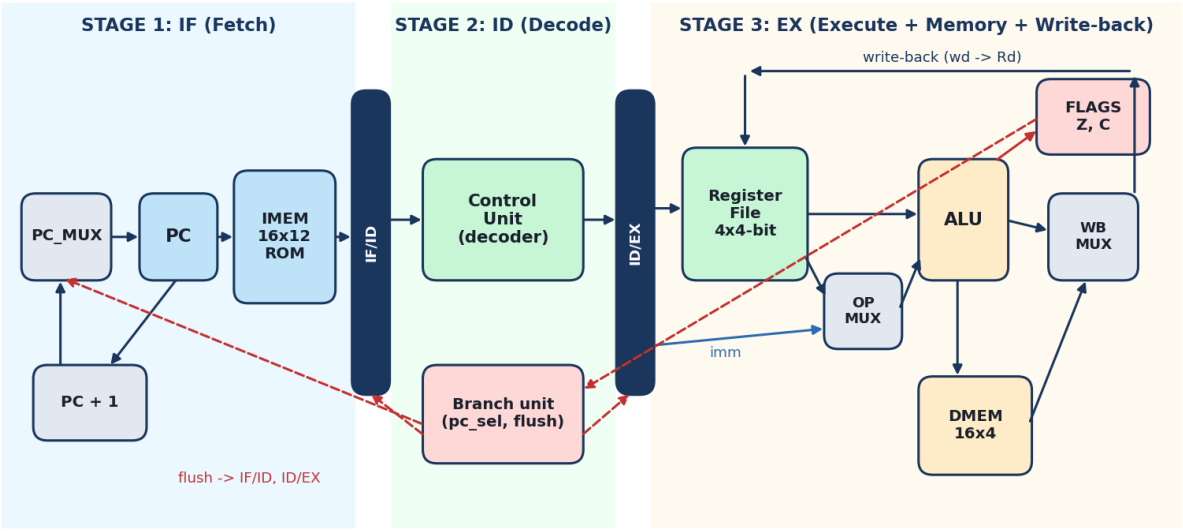


Figure 1. Conceptual datapath and pipeline partition derived from the RTL schematic (red dashed = branch / flush control)

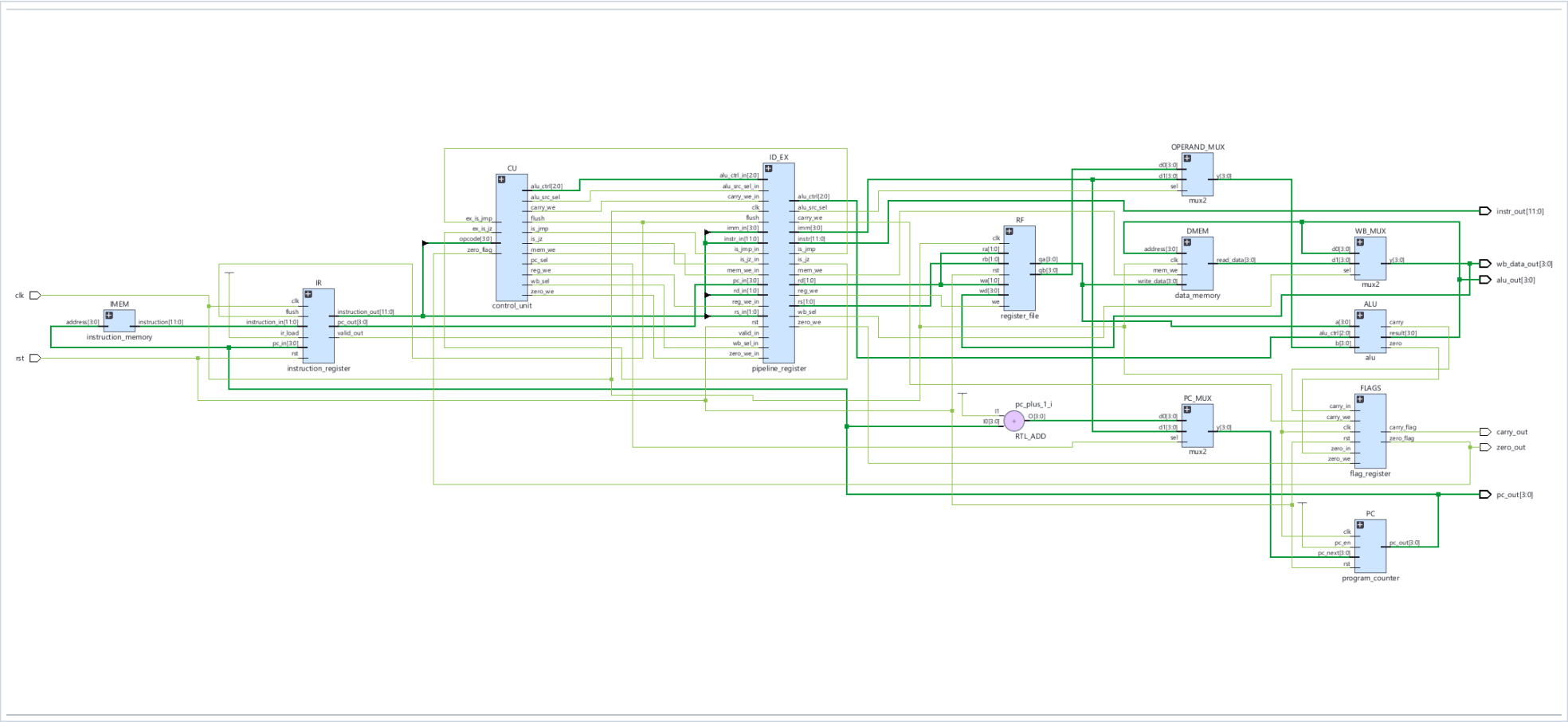


Figure 2. RTL Analysis schematic, top level (cpu_top): 13 instances wired as IF → IF/ID → ID → ID/EX → EX

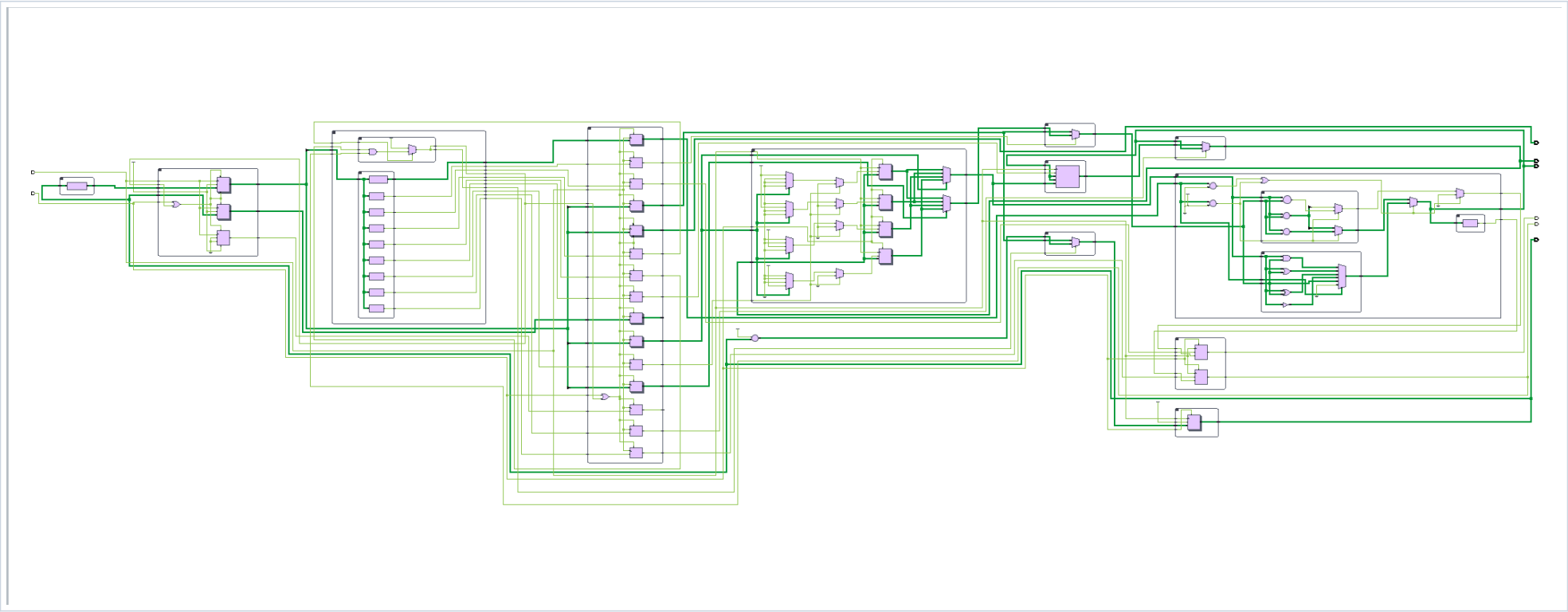


Figure 3. RTL Analysis schematic, expanded hierarchy: registers, adders, comparators and multiplexers inside each block

2.1 Functional blocks

Stage	Instance (module)	Type	Function and key ports
IF	PC (program_counter)	Sequential	4-bit state register; loads `pc_next[3:0]` each rising edge when `pc_en` = 1; synchronous reset to 0000
IF	pc_plus_1_i (RTL_ADD)	Combinational	Incrementer producing PC + 1, fed to PC_MUX input d0
IF	PC_MUX (mux2)	Combinational	Next-PC select: d0 = PC + 1, d1 = jump target (`imm[3:0]` from ID_EX); sel = `pc_sel`
IF	IMEM (instruction_memory)	Comb. ROM	16 × 12-bit; asynchronous read of `instruction[11:0]` at address = PC
IF/ID	IR (instruction_register)	Sequential	Latches `instruction_in[11:0]`, `pc_in[3:0]` and `valid_out`; `flush` turns it into a bubble
ID	CU (control_unit)	Combinational	Decodes opcode[3:0] into alu_ctrl[2:0], alu_src_sel, wb_sel, reg_we, mem_we, zero_we, carry_we, is_jmp, is_jz; branch unit uses ex_is_jmp, ex_is_jz, zero_flag to drive pc_sel and flush
ID/EX	ID_EX (pipeline_register)	Sequential	Carries the control word plus instr[11:0], pc[3:0], rd[1:0], rs[1:0], imm[3:0] into EX; cleared by flush or rst
EX	RF (register_file)	Mixed	4 × 4-bit registers; two combinational read ports (ra→qa, rb→qb), one clocked write port (wa, wd, we)
EX	OPERAND_MUX (mux2)	Combinational	ALU operand B: d0 = qb (Rs), d1 = imm; sel = alu_src_sel
EX	ALU (alu)	Combinational	4-bit adder/subtractor and logic unit; result[3:0], carry, zero under alu_ctrl[2:0]
EX	DMEM (data_memory)	Mixed	16 × 4-bit RAM; address = ALU result, write_data = qa (Rd); clocked write on mem_we, combinational read
EX	WB_MUX (mux2)	Combinational	d0 = ALU result, d1 = DMEM read_data; sel = wb_sel; drives RF wd and wb_data_out
EX	FLAGS (flag_register)	Sequential	Zero and Carry flip-flops, independently enabled by zero_we / carry_we

Table 2. Functional blocks identified in the RTL schematic

2.2 Datapath and control flow

1. **Fetch (IF).** PC addresses IMEM. The 12-bit word and the current PC are captured in IR at the next rising edge, while PC_MUX simultaneously loads PC + 1 (or a jump target) into PC.
2. **Decode (ID).** IR `instruction_out[11:8]` drives the CU opcode input. The decoder is purely combinational with safe defaults, so every opcode, including the four reserved ones, maps to a defined control word and no latches are inferred. The control word and register/immediate fields are registered in ID_EX.
3. **Execute (EX).** ID_EX `rd` and `rs` drive the RF read addresses directly. Operand A is qa (Rd); operand B comes from OPERAND_MUX. The ALU result either goes straight to WB_MUX or serves as the DMEM address. WB_MUX writes back to Rd at the end of the same cycle, and FLAGS capture Zero/Carry on the same edge.
4. **Next-PC resolution (EX → IF).** For JMP, or JZ with Zero = 1, the branch unit asserts `pc_sel` = 1 and `flush` = 1. At the next edge PC loads `imm[3:0]` and both IR and ID_EX are cleared, discarding the two wrongly fetched instructions.

2.3 Hazard handling at RTL

Hazard	Risk	Resolution in this design
Structural	Two stages need the same hardware in one cycle	Harvard organisation: IMEM serves IF while DMEM serves EX
Data (RAW)	A register is read before the previous instruction writes it	Registers are read in EX, one cycle after the producer wrote them; no forwarding or stall needed. Flags follow the same timing (SUB → JZ)
Control	Instructions fetched after a jump are wrong	Branch decided in EX; IF/ID and ID/EX flushed; exactly 2 bubbles per taken jump, 0 for a not-taken JZ

Table 3. Pipeline hazards and their RTL resolution

2.4 Observations from the expanded schematic

The expanded view (Figure 3) matches the intended micro-architecture. IR and ID_EX elaborate into banks of edge-triggered registers with a shared clear (flush OR rst). The register file elaborates into four 4-bit registers with write-address decoding and two 4:1 read multiplexers. The ALU contains adder/subtractor primitives, bitwise logic, a wide operation-select multiplexer and a zero-detect reduction, while DMEM is inferred as an RTL RAM primitive. The CU contains only comparators and gates, confirming fully combinational, hardwired control with no FSM.

3. Verification and Simulation Results

The CPU-level regression `tb_cpu` finished with **44 of 44 checks passed and 0 errors**, covering arithmetic, logic, memory, flags, jumps, pipeline flush and reset. The testbench is self-checking: it loads each program image, applies clock and reset, prints a per-cycle pipeline trace and compares architectural state (registers, flags, PC, data memory) against expected values.

Test	Program	Scope	Instr. / cycles	Checks
1	program.mem	Demonstration program, pipeline trace, JMP loop	5 in 7 (to XOR)	6 / 6
2	prog_arith.mem	ADD, SUB, STORE, LOAD, Zero / Carry	9 in 11	11 / 11
3	prog_logic.mem	AND, OR, XOR, NOT, MOV, LDI, Zero	12 in 14	8 / 8
4	prog_jump.mem	PC increment, JMP, JZ taken / not taken, flush	10 executed, 4 skipped	11 / 11
5	(reset)	Reset of PC, registers, flags, pipeline; X-check	—	8 / 8
Total				**44 / 44, errors = 0**

Table 4. Summary of the tb_cpu regression

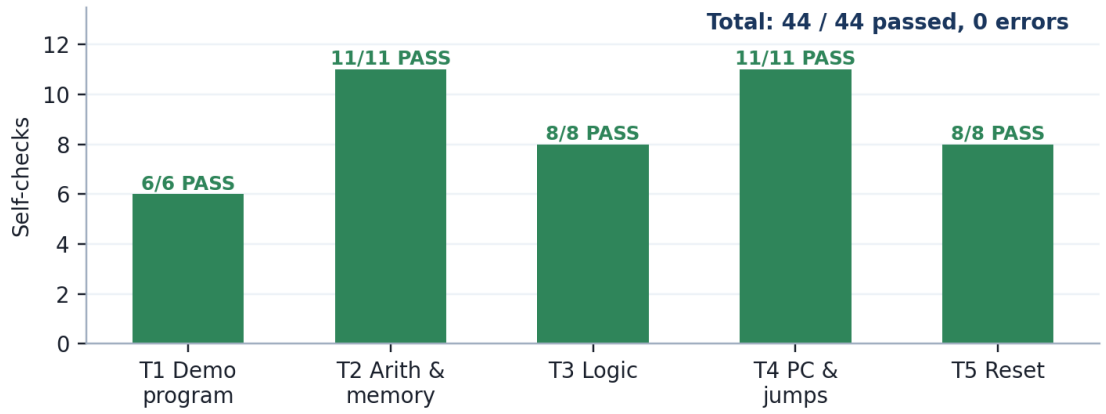


Figure 4. Self-checks passed per test scenario (counted from the Tcl console PASS lines)

Module-level results: `tb_alu` [11/11], `tb_register_file` [11/11], `tb_control_unit` [22/22] [attach console screenshots if required].

3.1 Waveform analysis – Test 1 (demonstration program)

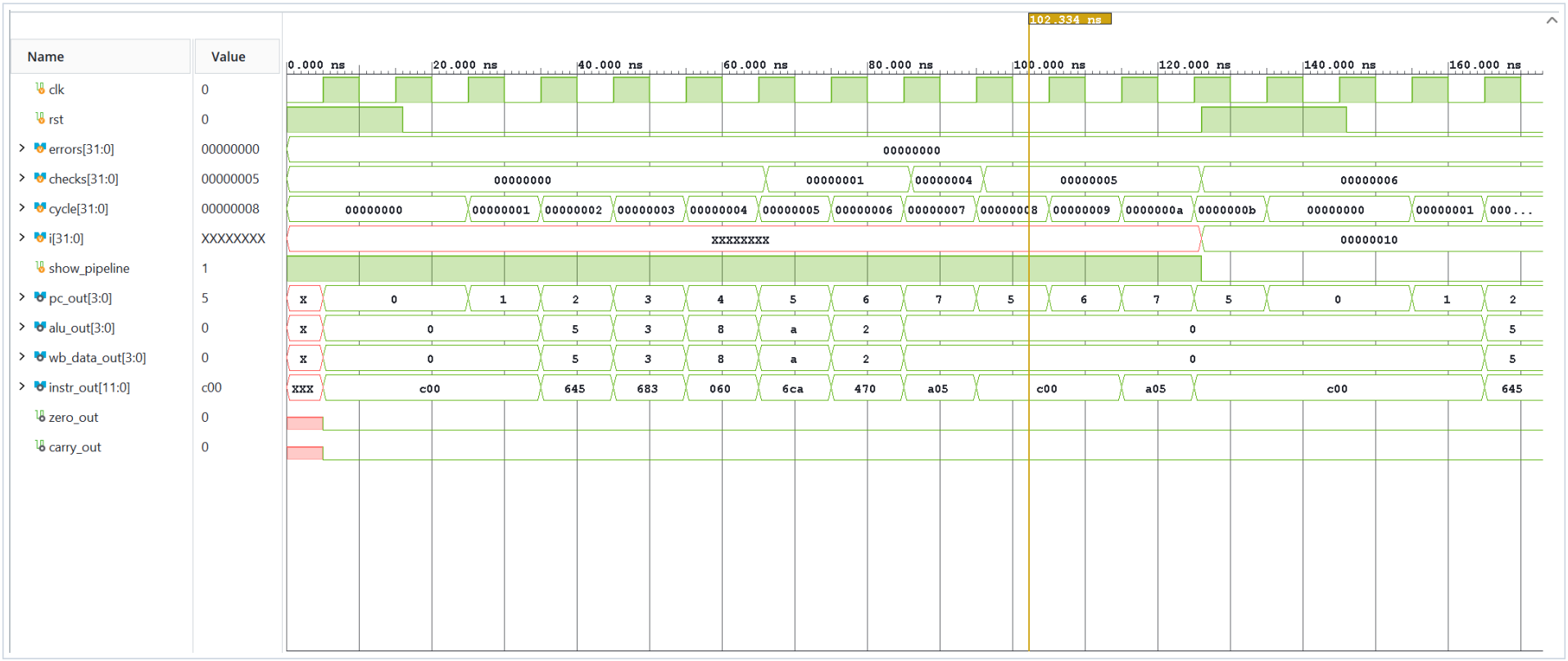


Figure 5. Behavioural simulation waveform of tb_cpu, 0–170 ns (cursor at 102.334 ns)

The waveform uses a 10 ns clock (first rising edge at 5 ns). The bus `instr_out[11:0]` is the instruction held in ID/EX, i.e. the instruction currently in EX. The following behaviour is observed in Figure 5:

- **Reset and X-resolution.** `rst` is held high until about 16 ns. `pc_out`, `alu_out`, `wb_data_out`, `instr_out`, `zero_out` and `carry_out` are X (red) only until the first rising edge at 5 ns, then resolve to PC = 0 and `instr_out` = 0xC00 (reserved opcode acting as a bubble). The synchronous reset initialises every state element.
- **Pipeline fill.** `pc_out` increments 0 → 1 → 2 … each cycle. When `pc_out` = 2 (35–45 ns), `instr_out` = 0x645, the instruction at address 0. This constant two-address offset is exactly the depth of a 3-stage pipeline.
- **Datapath results.** With one instruction completing per cycle, `alu_out` and `wb_data_out` step through 5 (LDI R1,5), 3 (LDI R2,3), 8 (ADD R1,R2), A (LDI R3,1010) and 2 (XOR R1,R3: 1000 XOR 1010 = 0010). The two buses are identical because `wb_sel` = 0 for these instructions.
- **Taken jump and flush.** 0xA05 (JMP 5) is in EX from 85 to 95 ns while `pc_out` = 7. At the 95 ns edge `pc_out` returns to 5 instead of 8, and `instr_out` shows 0xC00 for two cycles (95–115 ns): the two younger instructions were flushed. The cursor at 102.334 ns captures this state (`pc_out` = 5, `checks` = 5, `errors` = 0).
- **Second reset.** `rst` is re-asserted at about 126–146 ns for Test 5; PC returns to 0 and the program restarts (0x645, `alu_out` = 5 at about 165 ns).
- **Scoreboard.** `errors[31:0]` stays at 0x00000000 for the whole run, while `checks[31:0]` advances 0 → 1 → 4 → 5 → 6 in step with the console PASS lines.

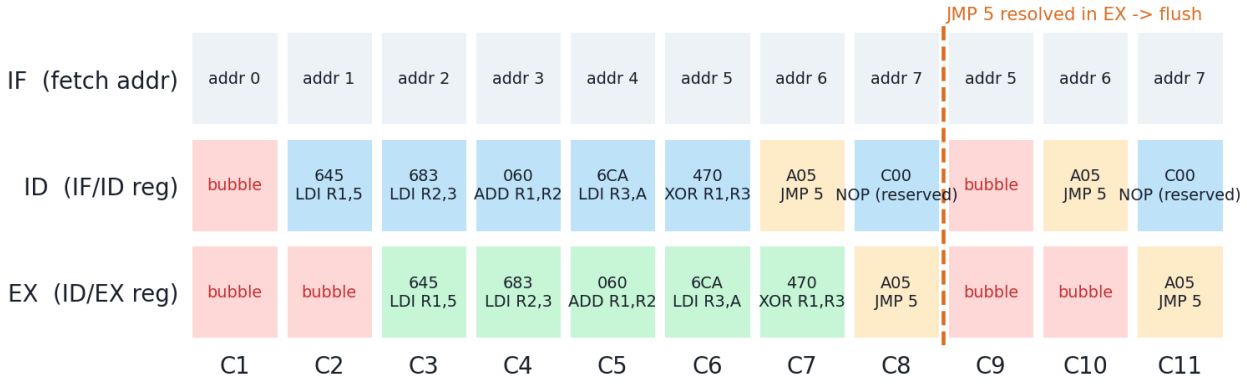


Figure 6. Pipeline occupancy for Test 1, reconstructed from the per-cycle console trace (C = clock cycle)

```
===== TEST 1: demonstration program (program.mem) =====
   cycle  1 | IF: addr  0 | ID: bubble          | EX: bubble          | R1=0000 R2=0000 R3=0000
   cycle  2 | IF: addr  1 | ID: 645 (addr  0) | EX: bubble          | R1=0000 R2=0000 R3=0000
   cycle  3 | IF: addr  2 | ID: 683 (addr  1) | EX: 645 (addr  0) | R1=0000 R2=0000 R3=0000
   cycle  4 | IF: addr  3 | ID: 060 (addr  2) | EX: 683 (addr  1) | R1=0101 R2=0000 R3=0000
   cycle  5 | IF: addr  4 | ID: 6ca (addr  3) | EX: 060 (addr  2) | R1=0101 R2=0011 R3=0000
PASS  ADD R1,R2  -> R1 (5+3) = 1000
   cycle  6 | IF: addr  5 | ID: 470 (addr  4) | EX: 6ca (addr  3) | R1=1000 R2=0011 R3=0000
   cycle  7 | IF: addr  6 | ID: a05 (addr  5) | EX: 470 (addr  4) | R1=1000 R2=0011 R3=1010
PASS  LDI R3,1010 -> R3 = 1010
PASS  XOR R1,R3  -> R1 = 0010
PASS  R2 still 0011 = 0011
      5 instructions finished after 7 clock cycles
   cycle  8 | IF: addr  7 | ID: c00 (addr  6) | EX: a05 (addr  5) | R1=0010 R2=0011 R3=1010
PASS  JMP 0101: PC (fetch address) = 5 = 0101
   cycle  9 | IF: addr  5 | ID: bubble          | EX: bubble          | R1=0010 R2=0011 R3=1010
   cycle 10 | IF: addr  6 | ID: a05 (addr  5) | EX: bubble          | R1=0010 R2=0011 R3=1010
   cycle 11 | IF: addr  7 | ID: c00 (addr  6) | EX: a05 (addr  5) | R1=0010 R2=0011 R3=1010
PASS  JMP runs again: program loops at 5 = 0101
```

Figure 7. Tcl console output – Test 1, demonstration program (program.mem)

Result – Test 1

ADD gives R1 = 1000 (5 + 3) at cycle 5; LDI and XOR give R3 = 1010 and R1 = 0010; R2 stays 0011. Five instructions complete in 7 cycles (5 + 2 fill). At cycle 8 the fetch address is redirected to 5, cycles 9–10 carry bubbles in ID/EX, and JMP executes again at cycle 11, proving a stable loop.

3.2 Test 2 – Arithmetic and data memory

```
===== TEST 2: arithmetic and memory (prog_arith.mem) =====
   EX addr  0: 645 | R0=0000 R1=0000 R2=0000 R3=0000 | Z=0 C=0
   EX addr  1: 683 | R0=0000 R1=0101 R2=0000 R3=0000 | Z=0 C=0
   EX addr  2: 060 | R0=0000 R1=0101 R2=0011 R3=0000 | Z=0 C=0
PASS  ADD R1,R2 (0101+0011) -> R1 = 1000
PASS  ADD: Carry = 0
PASS  ADD: Zero = 0
   EX addr  3: 940 | R0=0000 R1=1000 R2=0011 R3=0000 | Z=0 C=0
PASS  STORE R1,0 -> data_memory[0] = 1000
   EX addr  4: 645 | R0=0000 R1=1000 R2=0011 R3=0000 | Z=0 C=0
   EX addr  5: 160 | R0=0000 R1=0101 R2=0011 R3=0000 | Z=0 C=0
PASS  SUB R1,R2 (0101-0011) -> R1 = 0010
PASS  SUB: Zero = 0
PASS  SUB: Carry (1 = no borrow) = 1
   EX addr  6: 941 | R0=0000 R1=0010 R2=0011 R3=0000 | Z=0 C=1
PASS  STORE R1,1 -> data_memory[1] = 0010
   EX addr  7: 800 | R0=0000 R1=0010 R2=0011 R3=0000 | Z=0 C=1
PASS  LOAD R0,0 -> R0 = 1000
   EX addr  8: 1a0 | R0=1000 R1=0010 R2=0011 R3=0000 | Z=0 C=1
PASS  SUB R2,R2 -> R2 = 0000
PASS  Zero flag after 0000 result = 1
      9 instructions finished after 11 clock cycles (pipeline: 9 + 2)
```

Figure 8. Tcl console output – Test 2 (prog_arith.mem)

- **ADD R1,R2** (0101 + 0011) gives R1 = 1000 with Carry = 0 and Zero = 0.
- **STORE R1,0** (0x940) writes 1000 to data_memory[0], proving the immediate-address path through the ALU and the `mem_we` strobe.
- **SUB R1,R2** (0101 − 0011) gives 0010 with Zero = 0 and Carry = 1 (no borrow); **STORE R1,1** writes 0010 to data_memory[1].

- **LOAD R0,0** (0x800) returns 1000 into R0, confirming the `wb_sel` = 1 path through WB_MUX.
- **SUB R2,R2** gives 0000 and sets Zero = 1. Nine instructions retire in 11 cycles (9 + 2), confirming CPI = 1 after the fill.

3.3 Test 3 – Logic operations

```
===== TEST 3: logic operations (prog_logic.mem) =====
   EX addr  0: 64c | R0=0000 R1=0000 R2=0000 R3=0000 | Z=0 C=0
   EX addr  1: 68a | R0=0000 R1=1100 R2=0000 R3=0000 | Z=0 C=0
   EX addr  2: 7d0 | R0=0000 R1=1100 R2=1010 R3=0000 | Z=0 C=0
   EX addr  3: 2e0 | R0=0000 R1=1100 R2=1010 R3=1100 | Z=0 C=0
PASS  AND 1100,1010 -> R3 = 1000
   EX addr  4: 7d0 | R0=0000 R1=1100 R2=1010 R3=1000 | Z=0 C=0
   EX addr  5: 3e0 | R0=0000 R1=1100 R2=1010 R3=1100 | Z=0 C=0
PASS  OR  1100,1010 -> R3 = 1110
   EX addr  6: 460 | R0=0000 R1=1100 R2=1010 R3=1110 | Z=0 C=0
PASS  XOR 1100,1010 -> R1 = 0110
   EX addr  7: 790 | R0=0000 R1=0110 R2=1010 R3=1110 | Z=0 C=0
PASS  MOV R2,R1 (R1=0110) -> R2 = 0110
   EX addr  8: 605 | R0=0000 R1=0110 R2=0110 R3=1110 | Z=0 C=0
   EX addr  9: 500 | R0=0101 R1=0110 R2=0110 R3=1110 | Z=0 C=0
PASS  NOT 0101 -> R0 = 1010
   EX addr 10: 6ca | R0=1010 R1=0110 R2=0110 R3=1110 | Z=0 C=0
PASS  LDI R3,1010 -> R3 = 1010
   EX addr 11: 4f0 | R0=1010 R1=0110 R2=0110 R3=1010 | Z=0 C=0
PASS  XOR R3,R3 -> R3 = 0000
PASS  Zero flag after 0000 result = 1
      12 instructions finished after 14 clock cycles (pipeline: 12 + 2)
```

Figure 9. Tcl console output – Test 3 (prog_logic.mem)

With operands 1100 and 1010, AND gives 1000, OR gives 1110 and XOR gives 0110. MOV R2,R1 copies 0110, NOT 0101 gives 1010 in R0, and LDI R3,1010 followed by XOR R3,R3 clears R3 and sets Zero = 1. Carry stays 0 throughout, confirming that logic instructions never assert `carry_we`. Twelve instructions retire in 14 cycles (12 + 2).

3.4 Test 4 – Program counter and jumps

```
===== TEST 4: PC and jumps (prog_jump.mem) =====
PASS  PC after reset = 0000
PASS  PC increment (0 -> 1) = 0001
   EX addr  0: 601 | R0=0000 R1=0000 R2=0000 R3=0000 | Z=0 C=0
   EX addr  1: a04 | R0=0001 R1=0000 R2=0000 R3=0000 | Z=0 C=0
PASS  JMP 4: PC becomes 0100 = 0100
PASS  JMP flushed the ID/EX register = 0
   EX addr  4: 647 | R0=0001 R1=0000 R2=0000 R3=0000 | Z=0 C=0
   EX addr  5: 150 | R0=0001 R1=0111 R2=0000 R3=0000 | Z=0 C=0
PASS  SUB R1,R1 sets Zero = 1
   EX addr  6: b09 | R0=0001 R1=0000 R2=0000 R3=0000 | Z=1 C=1
PASS  JZ 9 taken: PC = 9 = 1001
   EX addr  9: 6c2 | R0=0001 R1=0000 R2=0000 R3=0000 | Z=1 C=1
   EX addr 10: 0f0 | R0=0001 R1=0000 R2=0000 R3=0010 | Z=1 C=1
PASS  ADD R3,R3 clears Zero = 0
   EX addr 11: b0e | R0=0001 R1=0000 R2=0000 R3=0100 | Z=0 C=0
PASS  JZ 14 not taken: next in EX is addr 12 = 1100
   EX addr 12: 686 | R0=0001 R1=0000 R2=0000 R3=0100 | Z=0 C=0
   EX addr 13: a0d | R0=0001 R1=0000 R2=0110 R3=0100 | Z=0 C=0
PASS  R0 = 1 (JMP skipped addresses 2, 3) = 0001
PASS  R2 = 6 (JZ skipped 7, 8; 14 never ran) = 0110
PASS  Program stops at PC = 13 = 1101
```

Figure 10. Tcl console output – Test 4 (prog_jump.mem)

- PC resets to 0000 and increments to 0001 on the next cycle.
- **JMP 4** at address 1 loads PC = 0100 and the ID/EX register is confirmed flushed (valid = 0); addresses 2 and 3 never execute, so R0 keeps 0001.
- **SUB R1,R1** sets Zero = 1 and the following **JZ 9** is taken (PC = 1001), proving that a flag written in one EX cycle is visible to the branch unit in the next.
- **ADD R3,R3** (0010 + 0010 = 0100) clears Zero; **JZ 14** is not taken and execution falls through to address 12 with no penalty.
- R2 = 0110 confirms that addresses 7, 8 and 14 were skipped; the program ends in a self-loop at PC = 13 (JMP 13, 0xA0D).

3.5 Test 5 – Reset and X-propagation

```
===== TEST 5: reset =====
PASS  PC after reset = 0000
PASS  R0 after reset = 0000
PASS  R2 after reset = 0000
PASS  Zero after reset = 0
PASS  Carry after reset = 0
PASS  IF/ID empty after reset = 0
PASS  ID/EX empty after reset = 0
   EX addr  0: 601 | R0=0000 R1=0000 R2=0000 R3=0000 | Z=0 C=0
   EX addr  1: a04 | R0=0001 R1=0000 R2=0000 R3=0000 | Z=0 C=0
PASS  no unknown (X) values on key outputs or registers

================================================
tb_cpu PASSED: 44 of 44 checks OK
================================================
```

Figure 11. Tcl console output – Test 5 (reset) and final regression summary

After a mid-run reset, PC, R0, R2, Zero and Carry all read 0 and both pipeline registers are empty. Execution restarts correctly (0x601, then 0xA04), and the final check confirms that no X values remain on any key output or architectural register.

3.6 Coverage and performance

Every one of the 12 instructions was executed and checked at least once. Both flags were verified in their set and cleared states, including the SUB no-borrow convention; both JZ outcomes, unconditional jumps, pipeline flush, back-to-back RAW dependencies and synchronous reset were exercised. The waveform signals transition exactly as the RTL intends, with no unexpected X/Z states or off-by-one pipeline timing.

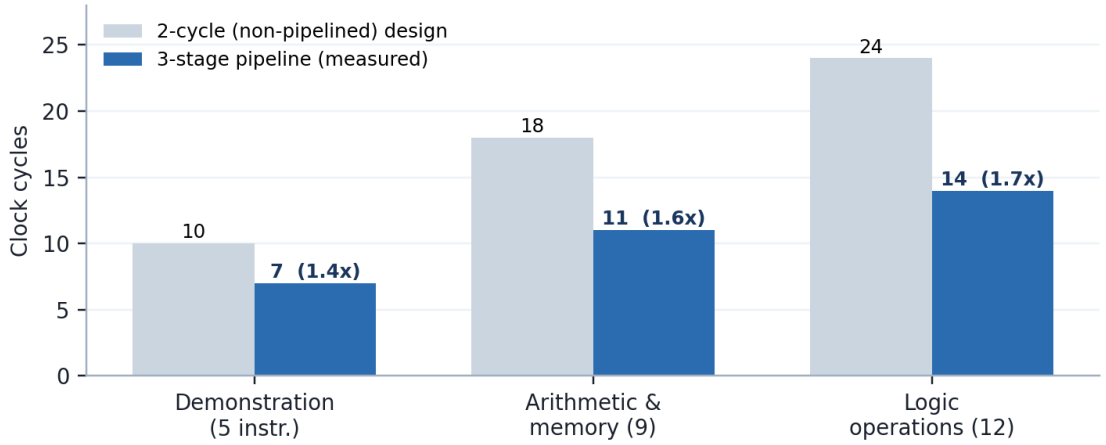


Figure 12. Clock cycles per program: 3-stage pipeline (measured in tb_cpu) vs. the earlier 2-cycle design

Result – Verification

44 / 44 checks passed, 0 errors. Measured CPI = 1 after a 2-cycle fill, giving a 1.4× to 1.7× speed-up over the 2-cycle design and approaching 2× for long jump-free code.

4. Resource Utilization

The implemented design on xc7a35tcpg236-1 occupies **41 LUTs, 58 flip-flops and 14 slices**, all below 0.2 % of the device, with no Block RAM, DSP or LUT-RAM. The only notable consumer is I/O: 28 bonded IOBs (26.4 %), driven by the debug buses exported at the top level.

4.1 Resource utilization

Resource	Used	Available	Utilization
Slice LUTs (all as logic)	41	20,800	0.20 %
LUT as distributed memory	0	—	0 %
Slice registers (flip-flops)	58	41,600	0.14 %
Slices	14	8,150	0.17 %
Block RAM tiles	0	50	0 %
DSP48E1	0	90	0 %
Bonded IOB	28	106	26.4 %
BUFGCTRL	1	32	3.1 %

Table 5. Post-implementation utilization (Vivado report utilization_3)

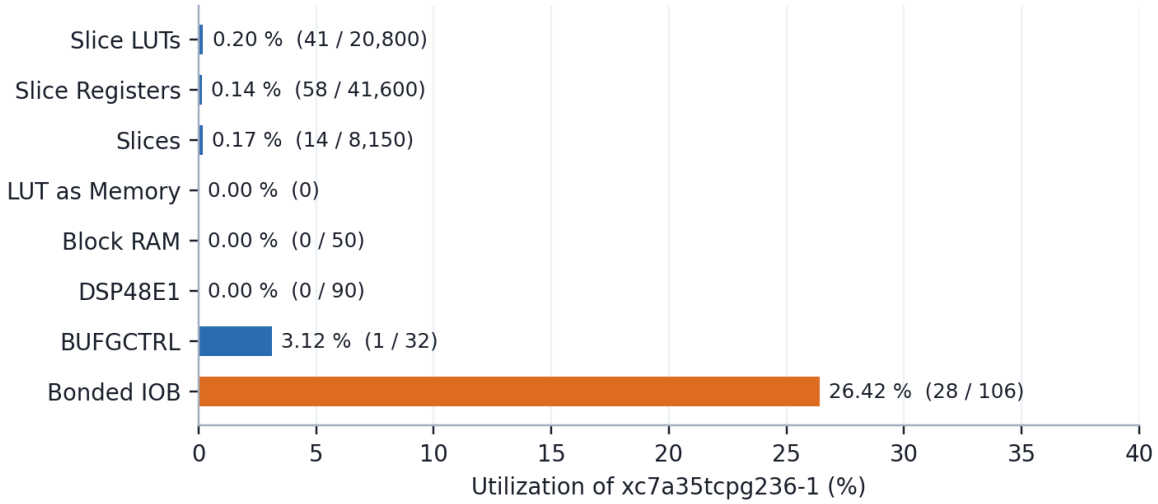


Figure 13. Device utilization by resource type (orange = resources above 5 %)

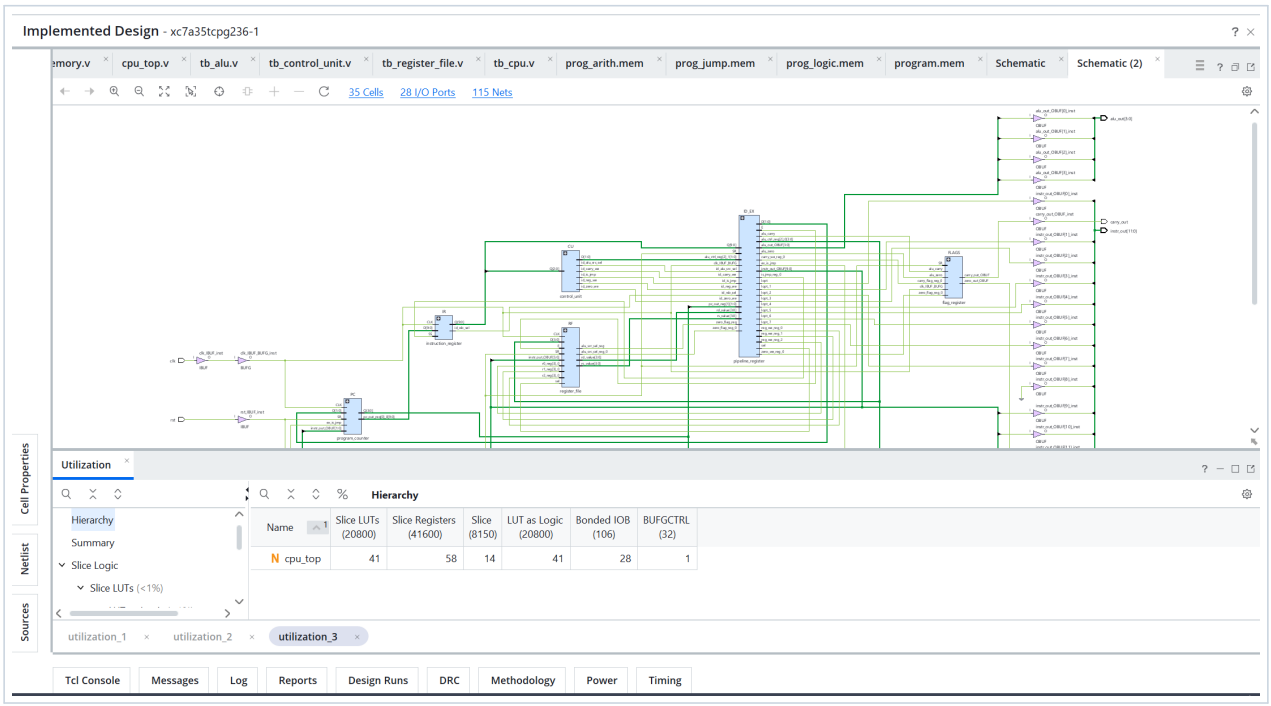


Figure 14. Vivado Utilization report, hierarchy view for cpu_top

The 28 IOBs match the port list exactly: `clk` and `rst` (2 IBUF), plus `pc_out[3:0]`, `alu_out[3:0]`, `wb_data_out[3:0]`, `instr_out[11:0]`, `zero_out` and `carry_out` (26 OBUF). The single BUFG distributes `clk` on a dedicated global net, so the design has one clock domain with no gated or derived clocks. Zero DSP usage is expected, since a 4-bit adder maps onto LUTs and the carry chain more efficiently than a DSP48. Zero BRAM is likewise correct, because both memories are far below the 18 Kb BRAM granularity.

4.2 Implemented netlist structure

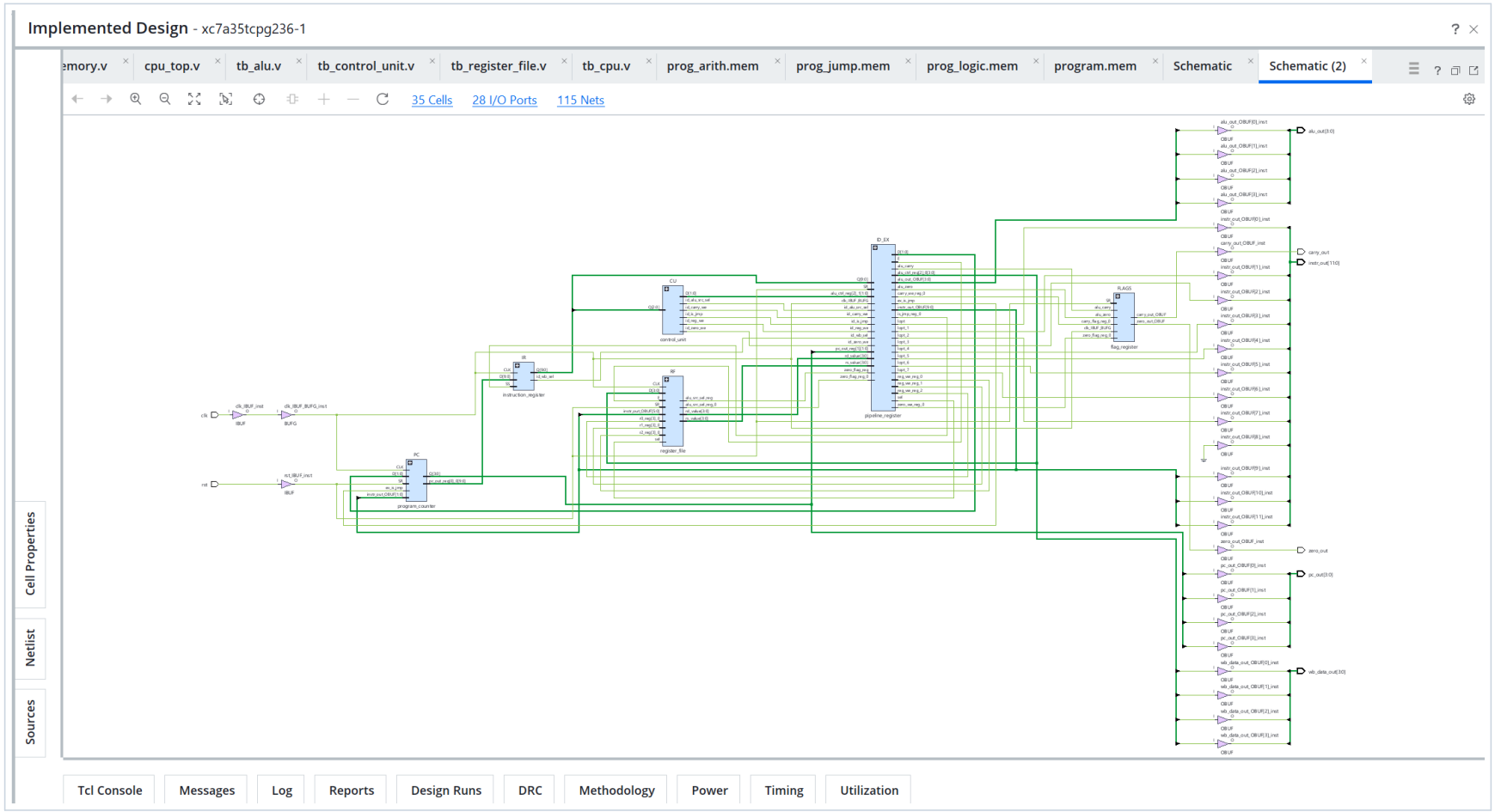


Figure 15. Implemented Design schematic: 35 cells, 28 I/O ports, 115 nets

The implemented schematic contains 35 cells: 2 IBUF, 26 OBUF, 1 BUFG and 6 hierarchical blocks (PC, IR, CU, RF, ID_EX, FLAGS). Synthesis has flattened IMEM, ALU, the three multiplexers and the PC incrementer into LUTs inside neighbouring blocks, and DMEM no longer appears at all. This is the result of aggressive constant propagation, and it matters when interpreting the numbers:

- **The instruction memory is a ROM initialised from a .mem file.** Vivado treats the program as a constant and folds it into the next-state logic. The IR and ID_EX instruction paths are registered as `Q[9:0]` instead of 12 bits, and `instr_out[8]` is tied directly to GND: every opcode in program.mem (0x6, 0x0, 0x4, 0xA and the 0xC bubble) is even, so bit 8 is provably constant.
- **Unused functions are trimmed.** program.mem contains no LOAD or STORE, so `mem_we` is never asserted and DMEM is optimised away (LUT as Memory = 0). ALU operations the program never selects are pruned in the same way.

Engineering note
The 41 LUT / 58 FF figure is the cost of the CPU **specialised to [program.mem – confirm the .mem file loaded at synthesis]**, not of the general-purpose processor. For a program-independent figure, preserve IMEM, DMEM and the ALU (KEEP_HIERARCHY / DONT_TOUCH, or make IMEM loadable from a port) and re-run synthesis. The full design is still expected to remain well under 1 % of the device.

4.3 Optimisation assessment

- **Logic efficiency.** Fixed-length decoding and a hardwired control word produce shallow, LUT-friendly logic with no FSM and no multi-cycle control.
- **Register budget.** Flip-flops are dominated by the pipeline registers (IR, ID_EX), the 16-bit register file and the 4-bit PC. This is the inherent cost of pipelining, and it buys CPI = 1 (Figure 12).
- **I/O pressure.** At 26.4 %, I/O is the only significant resource, and it comes entirely from observability ports. For a board-level demo these buses should be multiplexed onto LEDs or seven-segment displays, or observed with an Integrated Logic Analyzer (ILA).
- **Reset style.** The synchronous reset maps onto flip-flop SR/CE pins without extra LUTs, visible on the SR inputs of PC, ID_EX and FLAGS in Figure 15.

5. Conclusion

The 4-bit RISC CPU with a 3-stage IF/ID/EX pipeline is functionally correct and consistent from RTL description through simulation to resource utilization on the Artix-7 xc7a35tcpg236-1.

Level	Evidence	Status
RTL	Clean separation of hardwired, latch-free Control Unit and Harvard datapath; two pipeline registers bound three stages (Figures 1–3)	Met
Verification	tb_cpu 44 / 44 checks, 0 errors; all 12 instructions, flags, memory, JMP / JZ, flush, reset; CPI = 1 (Figures 4–12)	Met
Resource utilization	41 LUTs, 58 FFs, 14 slices (≤ 0.2 %), 0 BRAM / DSP, 1 global clock (Figures 13–15)	Met

Table 6. Summary of results by design level

Remaining work: [confirm the .mem file used in synthesis], re-synthesis with preserved memories for a program-independent utilization figure, and optionally a board-level demonstration on [board name]. Natural extensions include data forwarding with early branch resolution to remove the 2-cycle jump penalty, a wider PC for larger programs, and new instructions (shifts, compare, CALL/RET) in the four reserved opcodes.